EXPERIMENT - 4

AIM:

Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

Dataset Name: Iris Flower Dataset
File Used: iris.csv

2. Dataset Description

The dataset contains 3 classes of 50 instances each, where each class refers to a type of iris plant. This is a Classification problem where we predict the species based on physical measurements.

- Type: Numerical
- Domain: Botany / Biology
- Target Variable: species (Setosa, Versicolor, Virginica)
- **Key Features:**

Feature	Description	Data Type
sepal_length	Length of the sepal in cm	Numerical
sepal_width	Width of the sepal in cm	Numerical
petal_length	Length of the petal in cm	Numerical
petal_width	Width of the petal in cm	Numerical
species	The flower species (Setosa, Versicolor, Virginica)	Target (Label)

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN)

The algorithm classifies a data point based on how its neighbors are classified. It uses distance metrics to find the "nearest" points.

Euclidean Distance Equation:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

Where:

- **n** is the number of features.
- **p** and **q** are two data points.
- **d(p, q)** is the distance between them.

Prediction (Majority Voting):

$$\hat{y} = \text{mode}\{y_1, y_2, \ldots, y_k\}$$

Where:

- **k** is the number of neighbors chosen.
- y_i is the class label of the ith nearest neighbor.

4. Algorithm Limitations

- **Computationally Expensive:** Since it is a "Lazy Learner," it doesn't learn a model. Instead, it performs a search through the entire dataset during every prediction, making it slow for large datasets.
- **Sensitivity to Scale:** If one feature has a much larger range than others (e.g., 1000 vs 0.1), it will dominate the distance calculation. Feature scaling is mandatory.
- **The "Curse of Dimensionality":** As the number of features increases, the "distance" between points becomes less meaningful.

5. Methodology / Workflow

1. **Load Dataset:** Import the Iris dataset using Seaborn or Pandas.
2. **Exploratory Data Analysis:** Use pairplots to visualize how species cluster based on features.
3. **Data Preprocessing:** Separate features (X) and target (y). Scale the features using StandardScaler.
4. **Data Splitting:** Divide data into 80% Training and 20% Testing sets.
5. **Model Training:** Initialize KNeighborsClassifier and fit it to the training data.
6. **Hyperparameter Tuning:** Use the "Elbow Method" to plot Error Rate vs. K-Value to find the optimal k.
7. **Evaluation:** Use Accuracy, Confusion Matrix, and Classification Report.

6. Performance Analysis

The model is evaluated based on its ability to correctly distinguish between the three iris species.

- **Accuracy Score:** Measures the percentage of correct predictions (usually $>95\%$ for this dataset).
- **Confusion Matrix:** Shows if the model is confusing "Versicolor" with "Virginica."
- **Error Rate Plot:** identifies the "Sweet Spot" for k where the model is neither overfitting nor underfitting.

Observation: Random Forest typically yields a higher accuracy ($\approx 95\% - 98\%$) compared to a single Decision Tree ($\approx 85\%$), as it reduces the variance of individual errors.

7. Hyperparameter Tuning

We use **GridSearchCV** to systematically evaluate multiple combinations of hyperparameters to find the optimal configuration for the KNN model.

- **n_neighbors**: We tune the value of k (e.g., [1, 3, 5, 7, 9, 11, 13, 15]) to balance the bias-variance tradeoff.
- **weights**: We test 'uniform' (all neighbors have equal vote) versus 'distance' (closer neighbors have more influence).
- **metric**: We evaluate different distance calculation methods, such as 'euclidean' and 'manhattan'.

Code:

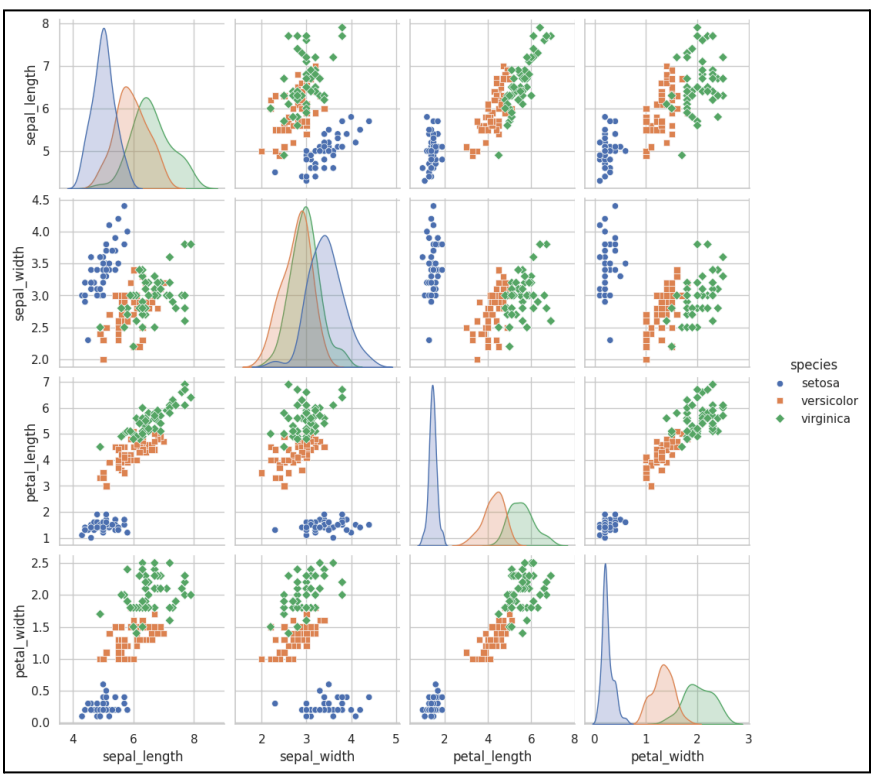
1. **Load and Explore Dataset**

```
iris = sns.load_dataset('iris')

print("Dataset Head:")

print(iris.head())
```

```
Dataset Head:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

2. **Visualize the relationship between features**

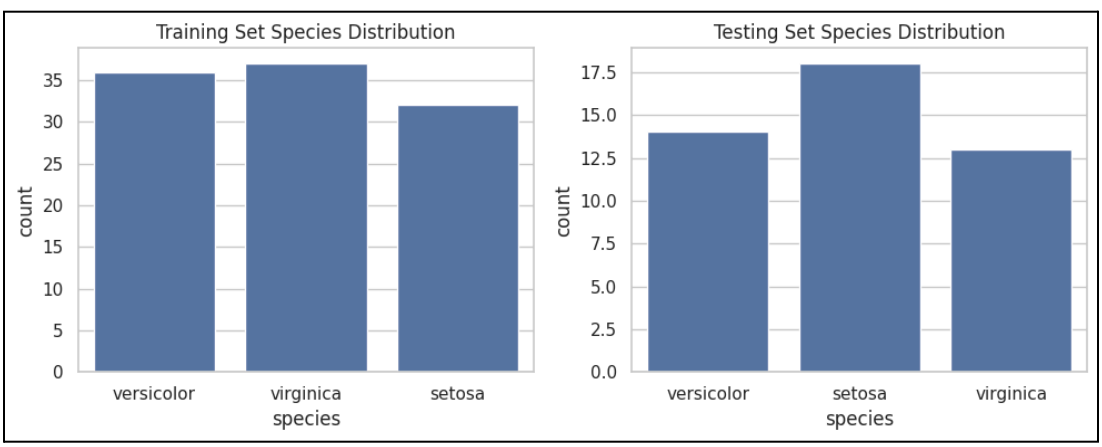
```
sns.pairplot(iris, hue='species', markers=["o", "s", "D"])

plt.show()
```

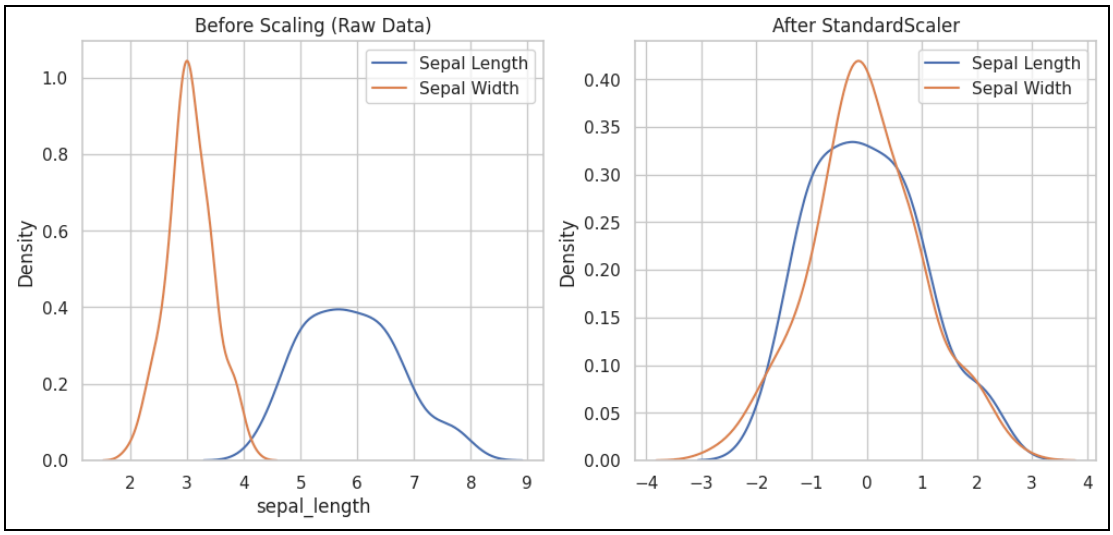


3. **Splitting the dataset for training and testing.**

```
# 1. Split features and target
X = iris.drop('species', axis=1)
y = iris['species']

# 2. Split into Training and Testing sets (70% train, 30% test)
# Note: Using your requested test_size=0.30 and random_state=60
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=60)

# --- VISUALIZATION: Class Distribution ---
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
sns.countplot(x=y_train)
plt.title('Training Set Species Distribution')

plt.subplot(1, 2, 2)
sns.countplot(x=y_test)
plt.title('Testing Set Species Distribution')
plt.tight_layout()
plt.show()

# 3. Feature Scaling (Normalization)
# We store unscaled data for comparison visualization
X_train_unscaled = X_train.copy()

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


# --- VISUALIZATION: Effect of Scaling ---

fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))


ax1.set_title('Before Scaling (Raw Data)')

sns.kdeplot(X_train_unscaled.iloc[:,0], ax=ax1, label='Sepal Length')

sns.kdeplot(X_train_unscaled.iloc[:,1], ax=ax1, label='Sepal Width')

ax1.legend()


ax2.set_title('After StandardScaler')

sns.kdeplot(X_train[:,0], ax=ax2, label='Sepal Length')

sns.kdeplot(X_train[:,1], ax=ax2, label='Sepal Width')

ax2.legend()

plt.show()
```





4. **Initializing the KNN model**

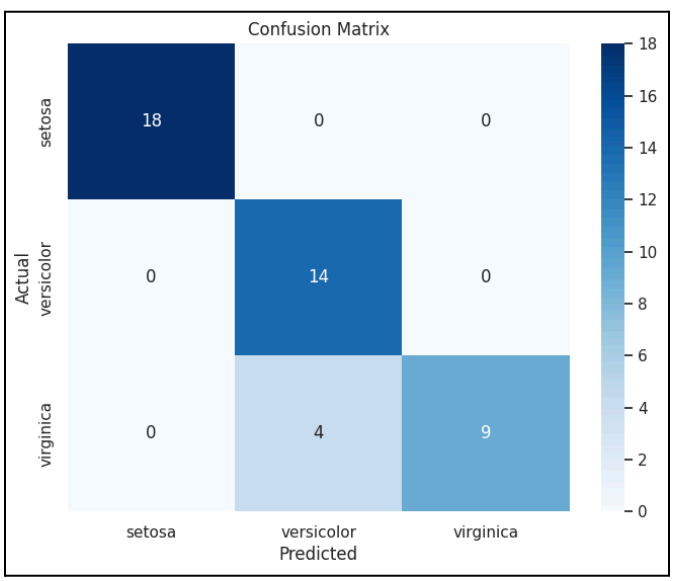
```
# Initialize the model

knn = KNeighborsClassifier(n_neighbors=6)

# Fit the model

knn.fit(X_train, y_train)
```

5. **Accuracy of the model using Confusion matrix**

```
# Make predictions

y_pred = knn.predict(X_test)


# Print Performance Metrics

print("Accuracy Score:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")

print(classification_report(y_test, y_pred))
```

```
# Plot Confusion Matrix
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues',
            xticklabels=knn.classes_, yticklabels=knn.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

```
Accuracy Score: 0.9111111111111111

Confusion Matrix:
[[18  0  0]
 [ 0 14  0]
 [ 0  4  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        18
  versicolor       0.78      1.00      0.88        14
   virginica       1.00      0.69      0.82        13

    accuracy                           0.91        45
   macro avg       0.93      0.90      0.90        45
weighted avg       0.93      0.91      0.91        45
```



6. Error rate v/s K Value

```
error_rate = []

# Check k values from 1 to 40

for i in range(1, 40):

    knn = KNeighborsClassifier(n_neighbors=i)

    knn.fit(X_train, y_train)

    pred_i = knn.predict(X_test)

    error_rate.append(np.mean(pred_i != y_test))


plt.figure(figsize=(10, 6))

plt.plot(range(1, 40), error_rate, color='blue', linestyle='dashed', marker='o',
markerfacecolor='red', markersize=10)

plt.title('Error Rate vs. K Value')

plt.xlabel('K')

plt.ylabel('Error Rate')

plt.show()
```

CONCLUSION

- **Decision Tree and Random Forest** were successfully implemented for the classification of heart disease.
- The **Random Forest** model generally outperformed the single Decision Tree in terms of accuracy and robustness.
- **Feature Importance** visualization allowed us to identify which medical attributes are most responsible for a high-risk diagnosis, making the model transparent and useful for clinical decision-making.